

برنامه نویسی پویا (Dynamic programming)

- یک روش حل مسائل بزرگ و سخت، تقسیم کردن آن به تعدادی نمونه مسئله کوچکتر و سپس پیدا کردن جواب آنهاست.
- در برخی مسائل، زیرمسئله ها مستقل از هم نیستند یعنی زیرمسئله های کوچکتر وابستگی و همپوشانی دارند.
- در اینگونه مسائل، روش تقسیم و غلبه ناکارآمد است. زیرا زیر مسئله ها کوچکتر که وابسته هستند به صورت تکراری محاسبه می گردند (مثلاً الگوریتم بازگشتی سری فیبوناچی)
- برنامه نویسی پویا همانند تقسیم و غلبه، مسئله را با ترکیب جواب های به دست آمده برای زیرمسئله ها حل می کند.

روش تقسیم و حل برای الگوریتم سری فیبوناچی

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ... **سری فیبوناچی**

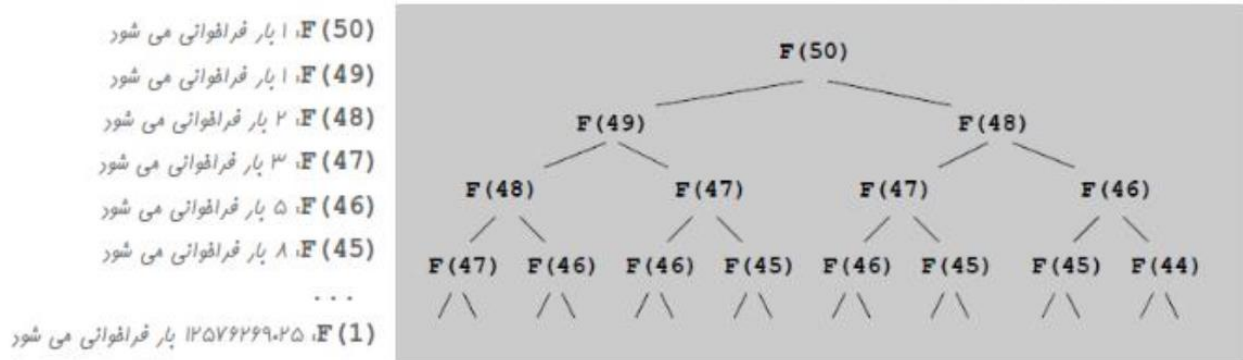
حل بازگشتی سری فیبوناچی

```
public static long F(int n)
{
    if (n == 0) return 0;
    if (n == 1) return 1;
    return F(n - 1) + F(n - 2);
}
```

پیچیدگی زمانی

$$F(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n \geq 2 \end{cases}$$

مثال: آیا راه حل بازگشتی برای محاسبه جمله شماره پنجاه سری فیبوناچی $F(50)$ مناسب است؟



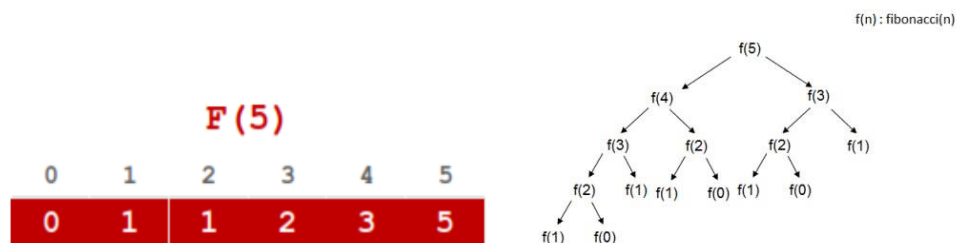
برای حل این مشکل ما می توانیم مقادیر تکراری (زیر مسئله های مشترک) را حل کرده و پاسخ آنها را ذخیره نموده و در صورت نیاز به حل مجدد زیرمسئله های مشترک از پاسخ ذخیره شده استفاده کنیم.

```
public static long F(int n)
{
    if (n == 0) return 0;
    long[] F = new long[n + 1];
    F[0] = 0;
    F[1] = 1;
    for (int i = 2; i <= n; i++)
        F[i] = F[i - 1] + F[i - 2];
    return F[n];
}
```

این کد تنها با ۴۹ عمل جمع می تواند جمله شماره پنجاه را محاسبه کند

برنامه نویسی پویا، از این لحاظ که نمونه به نمونه های کوچکتر تقسیم می شود، مشابه روش تقسیم و حل است ولی در این روش، نخست نمونه های کوچک تر حل می شود، نتایج ذخیره می گردد و هر گاه به یکی از آن ها نیاز پیدا شد، به جای محاسبه دوباره کافی است بازیابی گردد

برای مثال جمله شماره ۵



الگوریتم های تقسیم و غلبه مساله را به زیر مسائل تقسیم می کند و زیر مسائل را به صورت بازگشتی حل می کنند و سپس جواب زیر مساله ها را با هم ترکیب می کند تا جواب مساله کلی به دست آید. اما برنامه ریزی پویا زمانی کاربرد دارد که زیر مساله ها با هم همپوشانی دارد یعنی وقتی زیرمساله ها خود زیر مسائل مشترک دارند.

روش تقسیم و حل به دلیل حل مکرر زیرمسئله های مشترک در برخی از مسائل مانند سری فیبوناچی ناکارآمد است. اما روش برنامه نویسی پویا با یک رویکرد پایین به بالا، ابتدا زیرنمونه های کوچکتر را حل کرده و جواب آنها را ذخیره می کند.

برنامه نویسی پویا را می توان به ۴ مرحله تقسیم کرد:

- **توصیف ساختار یک جواب بهینه:** در این مرحله، مسئله را به زیرمسئله های کوچکتر و هم گونه تجزیه می کنیم و نشان می دهیم که جواب بهینه مسئله اصلی چگونه از جواب های بهینه زیرمسئله ها ساخته می شود. برای مثال فرض کنید یک مسئله $F(n)$ به دو زیرمسئله $F1$ و $F2$ تقسیم شود. آن گاه باید نشان دهیم در صورتی مسئله F بهینه است که جواب زیرمسئله های $F1$ و $F2$ بهینه باشد. این یک ویژگی کلیدی به نام **زیرساختار بهینه (Optimal Substructure)** است.
- **تعیین مقدار جواب بهینه به صورت بازگشتی:** یک رابطه بازگشتی ریاضی برای مقدار جواب بهینه بر اساس مقادیر زیرمسئله ها می نویسیم. این رابطه، همان **معادله بازگشت (Recurrence Relation)** است.
- **محاسبه مقدار یک جواب بهینه به سبک پایین به بالا:** این مرحله، هسته اصلی کارایی برنامه نویسی پویاست. به جای محاسبه بازگشتی که منجر به محاسبات تکراری فراوان می شود (مثلاً در محاسبه سری فیبوناچی $F(3)$ یا $F(1)$) بارها محاسبه می شود)، مقادیر را از کوچکترین زیرمسئله به سمت مسئله اصلی می سازیم و ذخیره می کنیم.
- **ساختن یک جواب بهینه از اطلاعات محاسبه شده:** در برخی مسائل (مانند کوله پشتی، کوتاه ترین مسیر)، این مرحله ضروری است، زیرا ما نه تنها به مقدار جواب (مثلاً بیشترین ارزش یا کوتاه ترین فاصله)، بلکه به **خود جواب** (مثلاً کدام آیتم ها را برداریم یا مسیر چیست) نیاز داریم.

مسئله برش بهینه یک میله

فرض کنید می خواهیم یک میله فولادی به طول n را برش بزیم. مشتری های مختلف برای خرید قطعات کوچک با اندازه های متفاوتی از این میله وجود دارند. اما قیمت قطعات کوچک تر دقیقاً بر اساس اندازه آن ها تعیین نمی شود.

در حالت کلی طول تمام قطعات یک عدد صحیح بر حسب متر و قیمت قطعات مختلف در یک آرایه $p[1..n]$ موجود است که $p[i]$ نشان دهنده ارزش یک قطعه با طول i می باشد هدف برش میله به گونه ای است که در نهایت بیشترین سود را داشته باشد

ورودی: یک طول به طول n و جدول قیمت های p_i برای $i = 1, 2, \dots, n$.

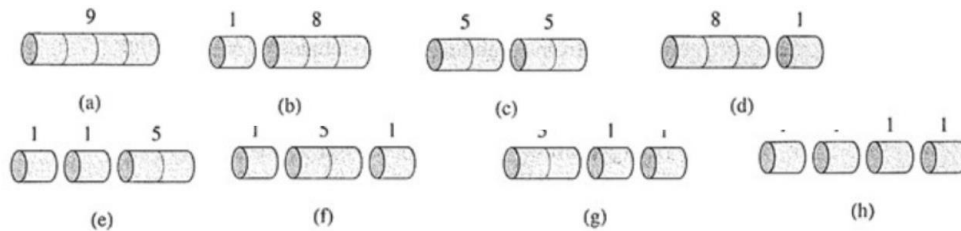
خروجی: بیشترین درآمد قابل دستیابی برای میله هایی که مجموع طول های آنها n است، محاسبه می شود.

اگر p_n به اندازه کافی بزرگ باشد، یک راه حل بهینه ممکن است نیاز به هیچ برشی نداشته باشد، یعنی میله را به طول n اینچ باقی بگذارید.

مثال: به طور مثال جدول زیر را در نظر بگیرید

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

اگر میله ای به طول ۴ داشته باشیم، به ۸ روش مختلف می توان برش زد:



می توانیم یک میله با طول n را به 2^{n-1} روش مختلف برش دهیم، چرا که برای هر نقطه ای در فاصله از انتهای چپ، برای $1, 2, \dots, n-1$ یک انتخاب مستقل مبنی بر برش یا عدم برش در آن نقطه داریم. بنابراین یک الگوریتم نمایی است

بهترین روش این است که آن را به دو قطعه ۲ اینچی برش دهیم و درآمد $p_2 + p_2 = 5 + 5 = 10$

اگر اولین برش در i انجام گیرد (یعنی اولین قطعه به طول i باشد) آنگاه سود بهینه برابر است با p_i به اضافه سود بهینه حاصل از برش یک قطعه بزرگ به اندازه $n-i$ می باشد. بنابراین اگر حداکثر سود حاصل از برش یک چوب به اندازه n را با r_n نمایش دهیم آنگاه می توانیم بنویسیم.

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

به عبارت ساده تر ما باید مقدار r_i را برای i های مختلف بدست آورده و با استفاده از آن مقدار بهینه کلی را پیدا کنیم. ما جدول زیر را برای مثال بدست می آوریم.

i	r_i	optimal solution
1	1	1 (no cuts)
2	5	2 (no cuts)
3	8	3 (no cuts)
4	10	2 + 2
5	13	2 + 3
6	17	6 (no cuts)
7	18	1 + 6 or 2 + 2 + 3
8	22	2 + 6

حال باید از بین تمام حالت های مختلف، حالت بهینه (در این جا بیشترین سود) را پیدا کنیم. لیستی از تمام حالت های ممکن در زیر آورده شده است و ما از بین آنها، بهینه ترین را انتخاب خواهیم کرد

$$\begin{aligned}
 & p_n: \text{درآمد حاصل از عدم برش,} \\
 & r_1 + r_{n-1}: \text{بیشترین درآمد از یک میله ۱ اینچی و یک میله } n-1 \text{ اینچی,} \\
 & r_2 + r_{n-2}: \text{بیشترین درآمد از یک میله ۲ اینچی و یک میله } n-2 \text{ اینچی, } \dots \\
 & \quad \cdot \\
 & \quad \cdot \\
 & \quad \cdot \\
 & r_{n-1} + r_1
 \end{aligned}$$

یعنی به عبارت دیگر:

$$r_n = \max \{p_n, r_1 + r_{n-1}, r_2 + r_{n-2}, \dots, r_{n-1} + r_1\}$$

یافتن زیرمسئله های بهینه: برای حل مسئله اصلی با اندازه n ، زیرمسائل با اندازه های کوچکتر را حل کنید. پس از انجام یک برش، دو زیرمساله باقی می ماند. راه حل بهینه برای مسئله اصلی شامل راه حل های بهینه برای زیرمسائل است. ممکن است زیرمسائل را مستقل از هم حل کنید.

مثال: برای $n = 7$ ، یکی از راه‌حل‌های بهینه ایجاد برش در ۳ اینچ است. این برش مسئله اصلی را به دو زیر مسئله با طول‌های ۳ و ۴ ایجاد می‌کند. باید هر دو را بهینه حل کنید.

راه‌حل بهینه برای مسئله طول ۴، برش به ۲ قطعه، هر کدام به طول ۲، که از آن در راه‌حل بهینه مسئله اصلی با طول ۷ استفاده می‌شود.

یک روش ساده برای شکستن مسئله به زیرمسئله

در این روش به جای آنکه هر مسئله را به دو زیر مسئله بشکنیم، هر مسئله را به یک زیرمسئله با اندازه کوچکتر تبدیل می‌کنیم.

هر راه‌حل بهینه حتما دارای یک سمت چپ‌ترین برش است. به عبارت دیگر، برشی وجود دارد که یک قطعه اول به طول i از انتهای چپ جدا می‌کند و یک قطعه باقیمانده به طول $n - i$ در سمت راست باقی می‌گذارد. به عبارت ساده‌تر فرض کنید در تقسیم‌بندی بهینه طول سمت چپ‌ترین قسمت i باشد.

- می‌توان گفت ما موفق شدیم اولین نقطه برش که سمت چپ‌ترین هم است، را پیدا کنیم. حالا کافی است قسمت باقیمانده (که طول آن $n-i$ است) را بصورت بهینه تقسیم بندی کنیم.

- پس ما به جای دو زیرمسئله کافی است، یک زیرمسئله را حل کنیم. ما باید زیرمسئله قسمت باقیمانده را حل کنیم.

- اگر بخواهیم مسئله را با این روش به زیرمسئله تبدیل کنیم، آن گاه ممکن است با یک مشکل روبرو شویم.

○ ممکن است، راه حل بهینه‌ای هم وجود داشته باشد که در آن نیاز به انجام هیچ برشی نباشد. در این صورت

مقدار i را باید چند قرار دهیم؟ در این صورت کافی است مقدار $i=n$ قرار دهیم و طول باقیمانده را صفر در نظر

بگیریم. سود ما برای طول صفر قطعا صفر خواهد بود. ($r_0 = 0$)

- با توجه با این شیوه جدید تقسیم کردن، نسخه ساده‌تری از معادله برای r_n بصورت زیر خواهد بود:

$$r_n = \max\{p_i + r_{n-i} : 1 \leq i \leq n\}$$

راه‌حل بازگشتی بالا به پایین

ما معادله ساده‌تر را که برای r_n بدست آوردیم را با تابع $\text{CUT-ROD}(p, n)$ پیاده سازی می‌کنیم. با فراخوانی این تابع مقدار بازگشتی سود بهینه ما r_n خواهد بود.

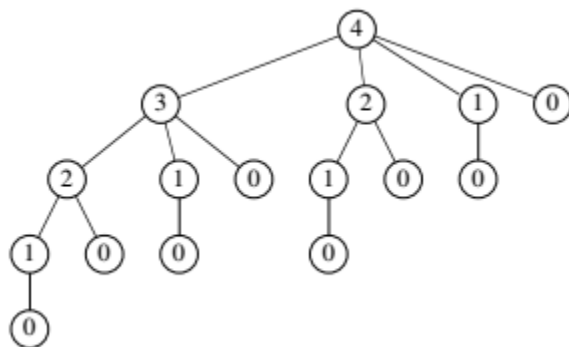
```

CUT-ROD( $p, n$ )
  if  $n == 0$ 
    return 0
   $q = -\infty$ 
  for  $i = 1$  to  $n$ 
     $q = \max\{q, p[i] + \text{CUT-ROD}(p, n - i)\}$ 
  return  $q$ 

```

این تابع پاسخ را برای ما پیدا می‌کند ، اما بسیار ناکارآمد است. اگر آن را کدنویسی و اجرا کنید، ممکن است برای $n = 40$ بیش از یک ساعت طول بکشد. تقریباً با هر یک واحد افزایش n زمان اجرا دو برابر می‌شود.

چرا اینقدر ناکارآمد؟ CUT-ROD به طور مکرر خود را فراخوانی می‌کند، حتی روی زیرمسائلی که قبلاً حل کرده است. در اینجا یک درخت از فراخوانی‌های بازگشتی برای $n = 4$ آورده شده است. داخل هر گره مقدار n برای فراخوانی نشان داده شده است:



درخت فراخوانی بازگشتی برای برش میله دارای تعداد زیادی زیرمساله تکراری است. زیرمساله برای اندازه ۲ را دو بار، برای اندازه ۱ را چهار بار و برای اندازه ۰ را هشت بار حل می‌کند.

رشد نمایی: پیچیدگی تابع CUT-ROD برابر است با:

$$T(n) = \begin{cases} 1 & \text{if } n = 0, \\ 1 + \sum_{j=0}^{n-1} T(j) & \text{if } n \geq 1. \end{cases}$$

زیگما تعداد فراخوانی‌هایی را می‌شمارد که این تعداد برابر $j = n - i$ است. حل این رابطه بازگشتی نشان می‌دهد که $T(n) = 2^n$ است.

همانطور که مشاهده کردید روش تقسیم و غلبه نمی‌تواند در حل این مسئله موفق عمل کند، زیرا زیرمسئله‌های مشترک زیادی وجود دارد. بنابراین بهتر است با استفاده از برنامه نویسی پویا مسئله را حل کنیم.

راه حل برنامه‌نویسی پویا برای مسئله برش میله

- به جای حل مکرر زیرمسائل‌های مشترک، زیرمساله را فقط یک بار حل کنید.
- راه‌حل یک زیرمساله را در یک جدول (آرایه) ذخیره کنید و از این به بعد هر زمان که به جواب زیرمساله مجدداً نیاز شد، به جدول مراجعه کنید.
- با ذخیره کردن جواب زیرمساله‌ها مصرف حافظه افزایش می‌یابد، اما در زمان صرف جویی می‌شود. پس باید به این دو منبع (حافظه - زمان) توازن ایجاد کرد.
- می‌تواند یک راه‌حل با زمان نمایی را به یک راه‌حل با زمان چندجمله‌ای تبدیل کند.
- دو روش اساسی: بالا به پایین یا یادآوری و پایین به بالا.

بالا به پایین به همراه حافظه

- به صورت بازگشتی حل کنید، اما هر نتیجه را در یک جدول ذخیره کنید.
 - برای یافتن راه‌حل یک زیرمساله، ابتدا در جدول جستجو کنید. اگر پاسخ وجود داشت، از آن استفاده کنید. در غیر این صورت، راه‌حل زیرمساله را محاسبه کنید و سپس راه‌حل را در جدول برای استفاده بعدی ذخیره کنید.
 - حافظه به معنای به خاطر سپردن چیزی است که قبلاً محاسبه شده است. ["Memoizing", "memorizing", "not"]
 - نسخه راه حل بازگشتی به همراه حافظه، راه حل زیرمساله برش مسئله بطول i را در خانه i ام آرایه r ذخیره می‌کند.
- $r[i]$

MEMOIZED-CUT-ROD(p, n)

```
let  $r[0:n]$  be a new array // will remember solution values in  $r$ 
for  $i = 0$  to  $n$ 
     $r[i] = -\infty$ 
return MEMOIZED-CUT-ROD-AUX( $p, n, r$ )
```

MEMOIZED-CUT-ROD-AUX(p, n, r)

```
if  $r[n] \geq 0$  // already have a solution for length  $n$ ?
    return  $r[n]$ 
if  $n == 0$ 
     $q = 0$ 
else  $q = -\infty$ 
    for  $i = 1$  to  $n$  //  $i$  is the position of the first cut
         $q = \max \{q, p[i] + \text{MEMOIZED-CUT-ROD-AUX}(p, n - i, r)\}$ 
 $r[n] = q$  // remember the solution value for length  $n$ 
return  $q$ 
```


روش حل پایین به بالا

زیرمسائل را بر اساس اندازه مرتب کنید و ابتدا کوچک‌ترین‌ها را حل کنید. به این ترتیب، هنگام حل یک زیرمساله، زیرمسائل کوچکتر را قبلاً حل کرده‌اید.

BOTTOM-UP-CUT-ROD(p, n)

```
let  $r[0 : n]$  be a new array      // will remember solution values in  $r$ 
 $r[0] = 0$ 
for  $j = 1$  to  $n$                   // for increasing rod length  $j$ 
     $q = -\infty$ 
    for  $i = 1$  to  $j$               //  $i$  is the position of the first cut
         $q = \max \{q, p[i] + r[j - i]\}$ 
     $r[j] = q$                     // remember the solution value for length  $j$ 
return  $r[n]$ 
```

بررسی زمان اجرا

هر دو نسخه بالا به پایین و پایین به بالا در زمان $\Theta(n^2)$ اجرا می‌شوند.

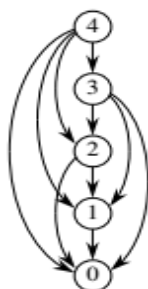
- **پایین به بالا:** دو حلقه تو در تو وجود دارد، که تعداد تکرارهای حلقه داخلی یک سری حسابی تشکیل می‌دهد.
- **بالا به پایین:** MEMOIZED-CUT-ROD هر زیرمساله را فقط یک بار حل می‌کند و زیرمسائل را برای اندازه‌های مختلف صفر تا n حل می‌کند. برای حل یک زیرمساله با اندازه n ، حلقه n بار تکرار می‌شود. در نتیجه، در تمام فراخوانی‌های بازگشتی، تعداد کل تکرارها یک سری حسابی تشکیل می‌دهد.

گراف زیرمسائل

گراف جهتدار راهکاری برای درک زیرمسئله‌ها و وابستگی‌های متقابل آنها به یکدیگر است.

- هر راس نشانه دهنده یک زیرمسئله متمایز است.
- یک یال جهت‌دار (x, y) وجود دارد اگر محاسبه یک راه‌حل بهینه برای زیرمساله x مستلزم دانستن یک راه‌حل بهینه برای زیرمساله y باشد.

مثال: برای مسئله برش میله با $n = 4$:



شکل ۱: گراف زیرمسائل برای برش میله

می‌توان گراف زیرمسائل را به عنوان نسخه فشرده‌ای از درخت فراخوانی‌های بازگشتی در نظر گرفت، که در آن تمام گره‌های مربوط به یک زیرمساله در یک رأس واحد فشرده می‌شوند و تمام یال‌ها از والد به فرزند می‌روند.

گراف زیرمسائل می‌تواند به تعیین زمان اجرا کمک کند. از آنجا که هر زیرمساله فقط یک بار حل می‌شود، زمان اجرا مجموع زمان‌های لازم برای حل هر زیرمساله است.

- زمان محاسبه راه‌حل برای یک زیرمساله معمولاً رابطه خطی بین درجه خروجی (تعداد یال‌های خروجی) رأس‌ها و تعداد یال‌ها است.

- تعداد زیرمسائل برابر تعداد رئوس است.

وقتی این شرایط برقرار باشد، زمان اجرا خطی نسبت به تعداد رئوس و یال‌ها است.

چگونه یک راه حل را بازسازی کنیم؟

تاکنون، بر محاسبه مقدار یک راه‌حل بهینه تمرکز کرده‌ایم، نه انتخاب‌هایی که منجر به یک راه‌حل بهینه شده‌اند.

روش پایین به بالا را گسترش دهید تا نه تنها مقادیر بهینه، بلکه انتخاب‌های بهینه را نیز ثبت کند. انتخاب‌های بهینه را در یک جدول جداگانه ذخیره کنید. سپس از یک روش جداگانه برای چاپ انتخاب‌های بهینه استفاده کنید.

```
EXTENDED-BOTTOM-UP-CUT-ROD( $p, n$ )
  let  $r[0:n]$  and  $s[1:n]$  be new arrays
   $r[0] = 0$ 
  for  $j = 1$  to  $n$  // for increasing rod length  $j$ 
     $q = -\infty$ 
    for  $i = 1$  to  $j$  //  $i$  is the position of the first cut
      if  $q < p[i] + r[j-i]$ 
         $q = p[i] + r[j-i]$ 
         $s[j] = i$  // best cut location so far for length  $j$ 
     $r[j] = q$  // remember the solution value for length  $j$ 
  return  $r$  and  $s$ 
```

اولین برش انجام شده در یک راه‌حل بهینه برای مسئله اندازه i را در $s[i]$ ذخیره می‌کند.

برای چاپ برش‌های انجام شده در یک راه‌حل بهینه:

```
PRINT-CUT-ROD-SOLUTION( $p, n$ )
  ( $r, s$ ) = EXTENDED-BOTTOM-UP-CUT-ROD( $p, n$ )
  while  $n > 0$ 
    print  $s[n]$  // cut location for length  $n$ 
     $n = n - s[n]$  // length of the remainder of the rod
```

مثال:

i	0	1	2	3	4	5	6	7	8
$r[i]$	0	1	5	8	10	13	17	18	22
$s[i]$		1	2	3	2	2	6	1	2

فراخوانی $\text{Print-Cut-Rod-Solution}(p, 8)$ Extended-Bottom-Up-Cut-Rod را برای محاسبه جداول r و s فوق فراخوانی می‌کند. سپس ۲ را چاپ می‌کند، n را به ۶ تنظیم می‌کند، ۶ را چاپ می‌کند و پایان می‌یابد (زیرا n به ۰ می‌رسد).

ضرب زنجیره‌ای ماتریس‌ها (Matrix-chain multiplication)

قبل از بیان مسئله چند نکته درباره ضرب زنجیره‌ای ماتریس‌ها بیان می‌کنیم

- در صورتی دو ماتریس A و B را می‌توانیم در هم ضرب کنیم که تعداد ستون‌های A با تعداد سطرهای B برابر باشد.

$$A_{p \times q} \times B_{q \times r} \Rightarrow C_{p \times r}$$

- ماتریس‌ها باید سازگار باشند. یعنی تعداد ستون‌های A برابر تعداد سطرهای B باشد.
- تعداد ضرب‌های اسکالر مورد نیاز برای محاسبه ماتریس C برابر است با: $p \times q \times r$
- ضرب ماتریس‌ها خاصیت جابه‌جایی ندارد، اما ضرب ماتریس‌ها خاصیت شرکت‌پذیری دارد:

$$(A_1 \times A_2) \times A_3 = A_1 \times (A_2 \times A_3)$$

بیان مسئله

دنباله‌ای از n ماتریس $\langle A_1, A_2, \dots, A_n \rangle$ موجود است. حاصلضرب $A_1 A_2 \dots A_n$ را با استفاده از ضرب استاندارد ماتریس (نه روش استراسن) محاسبه کنید بطوری که تعداد ضرب‌های اسکالر را به حداقل برسانید.

به عبارت دیگر، چگونه پرانتزگذاری را انجام دهیم تا تعداد ضرب‌های اسکالر به حداقل برسد؟

مثال: سه ماتریس $A_1: 10 \times 100, A_2: 100 \times 5, A_3: 5 \times 50$ موجود است و می‌خواهیم حاصلضرب $A_1 A_2 A_3$ را محاسبه کنید که یک ماتریس 10×50 است. با چه ترتیبی این سه ماتریس را در هم ضرب کنیم که کمترین تعداد ضرب اسکالر را داشته باشیم.

پاسخ: به دو حالت می‌توانیم این سه ماتریس را در هم ضرب کنیم.

- پرانتزگذاری به صورت $((A_1 A_2) A_3)$ را امتحان کنید. در این صورت ابتدا ماتریس‌های A_1 و A_2 در هم ضرب خواهند شد پس $10 \cdot 100 \cdot 5 = 5000$ ضرب اسکالر انجام می‌شود، سپس حاصلضرب این دو ماتریس در A_3 ضرب خواهد شد که با $10 \cdot 5 \cdot 50 = 2500$ ضرب اسکالر انجام می‌شود، که در مجموع 7500 می‌شود.
- پرانتزگذاری به صورت $(A_1 (A_2 A_3))$ را امتحان کنید. ابتدا $100 \cdot 5 \cdot 50 = 25,000$ ضرب اسکالر انجام دهید، سپس $10 \cdot 100 \cdot 50 = 50,000$ ، که در مجموع $75,000$ می‌شود.

روش اول ۱۰ برابر سریع‌تر است.

- $A_{P_0 \times P_1} A_{P_1 \times P_2} \dots A_{P_{n-1} \times P_n}$

P =

--	--	--	--	--	--	--	--	--	--

0 1 n

Diagram illustrating the mapping from positions to matrices. The top row shows positions $P_0, P_1, P_2, P_3, P_4, P_5$ with arrows pointing to the bottom row. The bottom row shows matrices M_1, M_2, M_3, M_4, M_5 . Arrows indicate that P_0 and P_1 map to M_1 , P_1 and P_2 map to M_2 , P_2 and P_3 map to M_3 , P_3 and P_4 map to M_4 , and P_4 and P_5 map to M_5 .

بنابراین پیچیدگی زمانی برابر تعداد حالت‌های پرانتزگذاری بوده و نمایی خواهد شد. در ادامه پیچیدگی این الگوریتم محاسبه شده است.

برای حالتی که $n \geq 2$ ، می‌توان مسئله را به دو قسمت تقسیم کرد. K را به عنوان نقطه تقسیم در نظر می‌گیریم. مقدار k می‌تواند از ۱ تا $n-1$ باشد. معادله بازگشت برای این مسئله بصورت زیر خواهد شد

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ \sum_{k=1}^{n-1} T(k)T(n-k) & \text{if } n \geq 2. \end{cases}$$

آنگاه تعداد حالات ممکن برای ضرب

$$T(n) = \sum_{i=1}^{n-1} T(i)T(n-i)$$

n	1	2	3	4	5	10	15
T(n)	1	1	2	5	14	4862	2674460

حل مسئله با استفاده از برنامه‌نویسی پویا

مرحله ۱: ساختار یک راه‌حل بهینه

حاصل ضرب ماتریس $A_i A_{i+1} \dots A_j$ را با $A_{i,j}$ نشان می‌دهیم.

اگر $j < i$ ، در این صورت مسئله به دو قسمت تقسیم می‌شود. بدست آورده حالت بهینه برای ضرب ماتریس‌ها از A_i تا A_k و بدست آوردن حالت بهینه برای ضرب ماتریس‌ها از A_{k+1} تا A_j . سپس ماتریس‌های بدست آمده از دو قسمت در هم ضرب خواهند شد. هزینه برابر است با

$$\text{cost of computing } A_{i,k} + \text{cost of computing } A_{k+1,j} + \text{cost of multiplying them together}$$

زیرساختار بهینه: فرض کنید پرانتزگذاری بهینه $A_{i,j}$ بین A_k و A_{k+1} تقسیم می‌شود. سپس پرانتزگذاری $A_{i,k}$ باید بهینه باشد. در غیر این صورت، اگر روش کم‌هزینه‌تری برای پرانتزگذاری آن وجود داشته باشد، از آن استفاده می‌کنید و یک پرانتزگذاری از $A_{i,j}$ با هزینه کمتر به دست می‌آورید. همین امر برای $A_{k+1,j}$ صادق است.

بنابراین، برای ساخت یک راه‌حل بهینه برای $A_{i,j}$ ، آن را به چگونگی پرانتزگذاری بهینه $A_{i,k}$ و $A_{k+1,j}$ تقسیم کنید، راه‌حل‌های بهینه را برای این زیرمسائل پیدا کنید و سپس راه‌حل‌های بهینه را ترکیب کنید. باید تمام تقسیم‌های ممکن را در نظر بگیرید.

زیرساختار بهینه: فرض کنید پرانتزگذاری بهینه $A_{i,j}$ بین A_k و A_{k+1} تقسیم می‌شود (یعنی k نقطه است که دنباله ماتریس‌ها را به دو قسمت تقسیم می‌کند). سپس پرانتزگذاری $A_{i,k}$ باید بهینه باشد. در غیر این صورت، اگر روش کم‌هزینه‌تری برای پرانتزگذاری آن وجود داشته باشد، از آن استفاده می‌کنید و یک پرانتزگذاری از $A_{i,j}$ با هزینه کمتر به دست می‌آورید. همین امر برای $A_{k+1,j}$ صادق است.

بنابراین، برای ساخت یک راه‌حل بهینه برای $A_{i,j}$ ، آن را به چگونگی پرانتزگذاری بهینه $A_{i,k}$ و $A_{k+1,j}$ تقسیم کنید، راه‌حل‌های بهینه را برای این زیرمسائل پیدا کنید و سپس راه‌حل‌های بهینه را ترکیب کنید. باید تمام تقسیم‌های ممکن را در نظر بگیرید.

مرحله ۲: یک راه‌حل بازگشتی

هزینه یک راه‌حل بهینه را به صورت بازگشتی بر اساس راه‌حل‌های زیرمسائل تعریف کنید.

اجازه دهید $m[i, j]$ حداقل تعداد ضرب‌های اسکالر برای محاسبه $A_{i,j}$ باشد. برای مسئله کامل، $m[1, n]$ را می‌خواهیم.

اگر $i = j$ ، فقط یک ماتریس وجود دارد $\Rightarrow m[i, i] = 0$ برای $i = 1, 2, \dots, n$.

اگر $j < i$ ، فرض کنید تقسیم بهینه بین A_k و A_{k+1} است، جایی که $k < j \leq i$.

بنابراین

$$m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j.$$

اما فرمول بالا فرض می‌کند که مقدار k را می‌دانید. پس باید تمام مقادیر ممکن را امتحان کنید و بهترین را انتخاب کنید،

$$\begin{aligned} & (M_1) * (M_2 * \dots * M_k * M_{k+1} * \dots * M_n) \\ & (M_1 * M_2) * (\dots * M_k * M_{k+1} * \dots * M_n) \\ & \dots \\ & (M_1 * M_2 * \dots * M_k * M_{k+1} * \dots) * (M_n) \end{aligned}$$

بنابراین

$$m[i, j] = \begin{cases} 0 & \text{if } i = j, \\ \min_{i \leq k < j} (m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j) & \text{if } i < j. \end{cases}$$

این فرمول هزینه یک راه‌حل بهینه را می‌دهد، اما نه نحوه ساخت آن. $S[i, j]$ را به عنوان مقداری از k تعریف کنید تا $A_{i,j}$ را در

یک پرانتزگذاری بهینه تقسیم کند. سپس $S[i, j] = k$ به طوری که $m[i, j] = m[i, k] + m[k+1, j] + p_{i-1}p_kp_j$

مرحله ۳: محاسبه هزینه‌های بهینه

می‌توان یک الگوریتم بازگشتی بر اساس معادله فوق برای $m[i, j]$ پیاده‌سازی کرد.

مشکل: زمان نمایی می‌برد.

زیرمسائل زیادی وجود ندارد: فقط یکی برای هر i, j به طوری که $1 \leq i \leq j \leq n$. $\binom{n}{2} + n = \Theta(n^2)$ از آنها وجود دارد.

بنابراین، یک الگوریتم بازگشتی همان زیرمسائل را بارها حل می‌کند.

به عبارت دیگر، این مسئله دارای زیرمسائل (مشترک) همپوشانی است.

در اینجا یک روش جدولی، پایین به بالا برای حل مسئله وجود دارد. زیرمسائل را به ترتیب افزایش طول زنجیره حل می‌کند. متغیر

$l = j - i + 1$ طول زنجیره را نشان می‌دهد.

```

MATRIX-CHAIN-ORDER( $p, n$ )
  let  $m[1:n, 1:n]$  and  $s[1:n-1, 2:n]$  be new tables
  for  $i = 1$  to  $n$  // chain length 1
     $m[i, i] = 0$ 
  for  $l = 2$  to  $n$  //  $l$  is the chain length
    for  $i = 1$  to  $n - l + 1$  // chain begins at  $A_i$ 
       $j = i + l - 1$  // chain ends at  $A_j$ 
       $m[i, j] = \infty$ 
       $m[i, j] = \infty$ 
      for  $k = i$  to  $j - 1$ 
         $q = m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$ 
        if  $q < m[i, j]$ 
           $m[i, j] = q$  // remember this cost
           $s[i, j] = k$  // remember this index
  return  $m$  and  $s$ 

```

تمام n زنجیره با طول ۱ به گونه‌ای مقداردی اولیه می‌شوند که $m[i, i] = 0$ برای $i = 1, 2, \dots, n$. سپس $n - 1$ زنجیره با طول ۲ محاسبه می‌شوند، سپس $n - 2$ زنجیره با طول ۳، و به همین ترتیب، تا ۱ زنجیره با طول n .

زمان: $O(n^3)$ ، از حلقه‌های سه‌گانه تو در تو. همچنین $\Omega(n^3) \Rightarrow \Theta(n^3)$.

مرحله ۴: ساخت یک راه‌حل بهینه

با جدول S پر شده، به صورت بازگشتی یک راه‌حل بهینه را چاپ کنید.

```

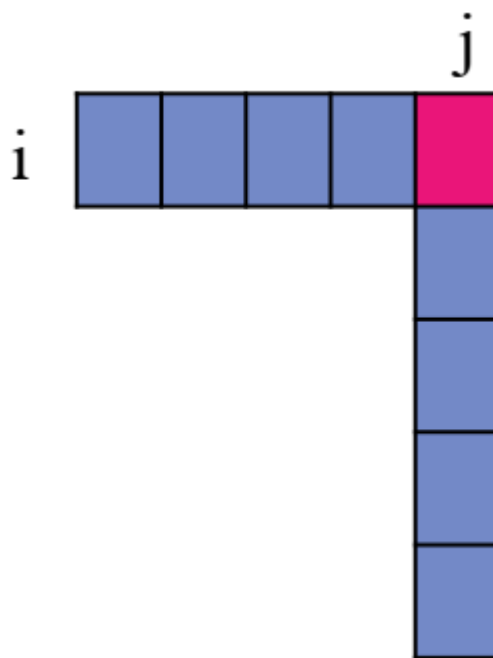
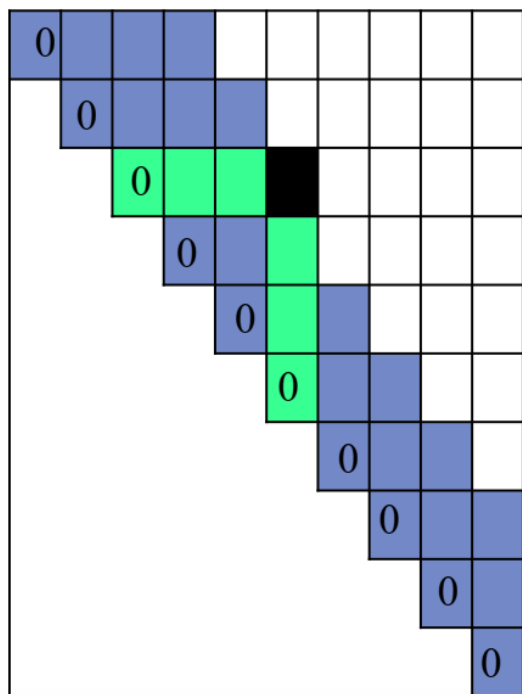
PRINT-OPTIMAL-PARENS( $s, i, j$ )
  if  $i == j$ 
    print " $A_i$ "
  else print "("
    PRINT-OPTIMAL-PARENS( $s, i, s[i, j]$ )
    PRINT-OPTIMAL-PARENS( $s, s[i, j] + 1, j$ )
  print ")"

```

فراخوانی اولیه Print-Optimal-Parens($s, 1, n$) است

پُر کردن ماتریس M

- ماتریس به صورت قطری پر می شود و کار از قطر اصلی با مقادیر صفر شروع می گردد. و سپس به مرور به سمت گوشه راست و بالای ماتریس پیش می رود
- هر عنصر $M[i][j]$ با استفاده از عناصر سمت چپ آن (موجود در سطر i و عناصر زیر آن (موجود در ستون j بدست خواهد آمد



- ضرب ماتریس ها با ابعاد $d_0=10, d_1=20, d_2=50, d_3=1, d_4=100$

$$M[i][j] = \min_{i \leq k \leq j-1} (M[i][k] + M[k+1][j] + d_{i-1} \times d_k \times d_j)$$

$$M[1,2] = \min_{1 \leq k < 2} \{M[1,k] + M[k+1,2] + d_0 d_k d_2\}$$

$$= d_0 d_1 d_2 = 10,000$$

$$M[2,3] = d_1 d_2 d_3 = 1,000$$

$$M[3,4] = d_2 d_3 d_4 = 5,000$$

قطر ۱

0			
	0		
		0	
			0

- ضرب ماتریس ها با ابعاد $d_0=10, d_1=20, d_2=50, d_3=1, d_4=100$

$$M[i][j] = \min_{i \leq k \leq j-1} (M[i][k] + M[k+1][j] + d_{i-1} \times d_k \times d_j)$$

$$M[1,3] = \min_{1 \leq k < 3} \{M[1,k] + M[k+1,3] + d_0 d_k d_3\}$$

$$= \min \{M[1,1] + M[2,3] + d_0 d_1 d_3,$$

$$M[1,2] + M[3,3] + d_0 d_2 d_3\}$$

$$= \min \{0 + 1000 + 200, 10000 + 0 + 500\}$$

$$= 1200$$

قطر ۲

0			
	0		
		0	
			0

- ضرب ماتریس ها با ابعاد $d_0=10, d_1=20, d_2=50, d_3=1, d_4=100$

$$M[i][j] = \min_{i \leq k \leq j-1} (M[i][k] + M[k+1][j] + d_{i-1} \times d_k \times d_j)$$

$$\begin{aligned} M[2,4] &= \min_{2 \leq k < 4} \{M[2,k] + M[k+1,4] + d_1 d_k d_4\} \\ &= \min \{M[2,2] + M[3,4] + d_1 d_2 d_4, \\ &\quad M[2,3] + M[4,4] + d_1 d_3 d_4\} \\ &= \min \{0 + 5000 + 100000, 1000 + 0 + 2000\} \\ &= 3000 \end{aligned}$$

قطر ۲

0			
	0		
		0	
			0

- ضرب ماتریس ها با ابعاد $d_0=10, d_1=20, d_2=50, d_3=1, d_4=100$

$$M[1][n] = M[1][k] + M[k+1][n] + d_0 d_k d_n$$

$$\begin{aligned} M[1,4] &= \min_{1 \leq k < 4} \{M[1,k] + M[k+1,4] + d_0 d_k d_4\} \\ &= \min \{M[1,1] + M[2,4] + d_0 d_1 d_4, \\ &\quad M[1,2] + M[3,4] + d_0 d_2 d_4, \\ &\quad M[1,3] + M[4,4] + d_0 d_3 d_4\} \\ &= 2200 \end{aligned}$$

قطر ۳

0			
	0		
		0	
			0

• چاپ ترتیب بهینه ضرب ماتریس ها

```

1. void order(index i, index j)
2. {
3.     if (i == j)
4.         cout << "A" << i;
5.     else
6.     {
7.         k = P[i][j];
8.         cout << "(";
9.         order (i, k);
10.        order (k + 1, j);
11.        cout << ") ";
12.    }
13. }

```

	1	2	3	4	5	6
1		1	1	1	1	1
2			2	3	4	5
3				3	4	5
4					4	5
5						5

$$T(n) \in \Theta(n)$$

$A_1 (A_2 A_3 A_4 A_5 A_6)$
 $(A_2 A_3 A_4 A_5) A_6$
 $A_1 ((A_2 A_3 A_4 A_5) A_6)$
 $A_1 (((A_2 A_3) A_4) A_5) A_6$

ماتریس P

- $P=[3, 6, 4, 2, 5]$

- $A_1: 3 \times 6$
- $A_2: 6 \times 4$
- $A_3: 4 \times 2$
- $A_4: 2 \times 5$

j

	1	2	3	4
1	0			
2		0		
3			0	
4				0

i

	1	2	3	4
1				
2				
3				
4				

m

s

	1	2	3	4
1	0	72		
2		0	48	
3			0	40
4				0

	1	2	3	4
1		1		
2			2	
3				3
4				

$P=[3, 6, 4, 2, 5]$

$$m[i, j] = \min_{i \leq k < j} (m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j)$$

$$m[1, 2] = m[1, 1] + m[2, 2] + p[0] * p[1] * p[2] = 72$$

$$m[2, 3] = m[2, 2] + m[3, 3] + p[1] * p[2] * p[3] = 48$$

$$m[3, 4] = m[3, 3] + m[4, 4] + p[2] * p[3] * p[4] = 40$$

	1	2	3	4
1	0	72	84	
2		0	48	108
3			0	40
4				0

	1	2	3	4
1		1	1	
2			2	3
3				3
4				

$P=[3, 6, 4, 2, 5]$, $L=3$

$$m[i, j] = \min_{i \leq k < j} (m[i, k] + m[k+1, j] + p_{i-1} p_k p_j)$$

$$m[1, 3] = \min \{ m[1, 1] + m[2, 3] + p[0] * p[1] * p[3], m[1, 2] + m[3, 3] + p[0] * p[2] * p[3] \}$$

$$= \min \{ 84, 96 \} = 84$$

$$m[2, 4] = \min \{ m[2, 2] + m[3, 4] + p[1] * p[2] * p[4], m[2, 3] + m[4, 4] + p[1] * p[3] * p[4] \}$$

$$= \min \{ 180, 108 \} = 108$$

	1	2	3	4
1	0	72	84	114
2		0	48	108
3			0	40
4				0

	1	2	3	4
1		1	1	3
2			2	3
3				3
4				

$P=[3, 6, 4, 2, 5]$, $L=4$

$$m[i, j] = \min_{i \leq k < j} (m[i, k] + m[k+1, j] + p_{i-1} p_k p_j)$$

$$m[1, 4] = \min \{ m[1, 1] + m[2, 4] + p[0] * p[1] * p[3],$$

$$m[1, 2] + m[3, 4] + p[0] * p[2] * p[4],$$

$$m[1, 3] + m[4, 4] + p[0] * p[3] * p[4] \} = \min \{ 144, 172, 114 \} = 114$$

	1	2	3	4
1		1	1	3
2			2	3
3				3
4				

A1 A2 A3 A4



(A1 A2 A3)(A4)



(A1 (A2 A3))(A4)